

中国矿业大学计算机学院

2023 级本科生课程报告

课程名称： 数据挖掘基础

报告题目： 分类算法原理与应用

报告时间： 2025.12.3

学生姓名： 王志豪

学 号： 08232337

专 业： 数据科学与大数据技术

任课教师： 祝汉城

课程报告成绩评定及评分依据

序号	评分依据	分值	实际得分
1	课程报告描述了数据挖掘技术或算法的基本原理	20	
2	课程报告详细描述了待解决的问题，如有可能参考教材给出解决问题的过程	30	
3	课程报告使用 Python 程序设计语言（附源代码）对待解决问题进行了实验，给出了相关实验结果或可视化结果	40	
4	课程报告给出了个人学习体会	10	
总分			
教师评语： 教师签名： 年 月 日			

目 录

1 绪论	1
2 问题描述	1
3 分类算法原理与优缺点分析	2
3.1 朴素贝叶斯	2
3.2 决策树	3
3.3 K 最近邻算法	4
4 源代码及实现结果	5
4.1 源代码分析	5
4.2 结果与分析	11
5 个人体会	13

1. 绪论

在众多分类算法中，基于机器学习的文本分类技术因其强大的泛化能力与自适应性而成为主流，其中朴素贝叶斯、决策树与 K 最近邻（KNN）作为监督学习领域的经典算法，各自具有独特的数学机理与适用场景，朴素贝叶斯基于概率统计理论，在处理高维文本数据时往往表现出惊人的效率；决策树通过构建层级化的规则结构，具有较强的可解释性；而 K 最近邻算法则依赖样本间的距离度量，直观且无需显式训练。为了深入探究这三种算法在实际垃圾短信识别任务中的性能差异，本实验选取了经典的 SMSSpamCollection 数据集作为研究对象，首先对原始文本数据进行预处理与 TF-IDF 特征提取，将其转化为计算机可识别的向量空间表示，随后分别构建三种分类模型进行训练与测试，本研究将从准确率、精确率、召回率、F1 分数以及模型的时间复杂度（训练与预测时间）等多个维度进行综合量化评估，并结合雷达图、ROC-AUC 曲线及 PR-AUC 曲线等可视化手段，深入分析各算法在应对类别不平衡数据时的鲁棒性与优劣势，旨在为构建兼顾高识别精度与实时响应速度的垃圾短信过滤系统提供实证依据与算法选型参考。

2. 问题描述

在本实验中，我们选用经典的 SMSSpamCollection 数据集作为研究对象，该数据集由数千条经过人工标注的短信息样本组成，通过 TF-IDF 技术预处理后转化为高维稀疏的文本向量特征，分别对应“正常短信（Ham）”与“垃圾短信（Spam）”两类标签，其数据结构具有两个显著特点：一是特征维度极高且稀疏，这对基于距离度量的算法构成了“维度灾难”的潜在挑战；二是存在显著的类别不平衡现象，正常短信样本数量远多于垃圾短信，对模型的查全率与查准率平衡提出了更高要求。基于此，为全面评估不同监督学习范式在面对高维文本分类任务时的表现，本实验从主流分类方法体系中各选取一个具有代表性的典型算法，包括概率生成模型中的朴素贝叶斯（Naive Bayes）、规则判别模型中的决策树（Decision Tree）以及基于实例的惰性学习算法 K 最近邻（KNN），并在统一的特征提取与划分标准下对其进行对比实验。各算法分别对应不同的数学机理与归纳偏置：朴素贝叶斯基于特征条件独立假设，往往在文本数据上表现出极高的计算效率与稳健性；决策树通过构建层级化的判断规则挖掘数据特征，具备较强的可解释性与非线性处理能力；KNN 则直接依赖特征空间中的距离计算进行分类，直观但易受高维

稀疏性影响。通过对这些多范式算法在垃圾短信数据集上的分类结果进行多维度量化评测（包括 Accuracy、Precision、Recall、F1-score、训练时间与预测时间）以及雷达图、ROC-AUC 曲线与 PR-AUC 曲线的可视化展示，本实验旨在系统分析不同算法在处理高维稀疏数据与类别不平衡问题时的性能差异、计算开销权衡与鲁棒性表现，从而为文本分类场景下的算法选型与优化提供理论依据与实证支持。

3. 分类算法原理与优缺点分析

3.1 朴素贝叶斯

该算法是基于贝叶斯公式得来的，其核心思想在于：通过统计每个特征在不同类别中出现的概率，计算“哪个类别最可能产生这组特征”，然后选择概率最大的类别。具体的计算思路：对于给定特征向量 $\mathbf{x} = (x_1, x_2, \dots, x_d)$ ，预测类别 c 时，朴素贝叶斯计算：

$$\hat{c} = \arg \max_{c \in \mathcal{C}} P(c) \prod_{i=1}^d P(x_i | c) \quad (3.1)$$

其中： $P(c)$ 是先验概率， $P(x_i | c)$ 是似然概率。简单来说就是要对一个新的记录进行分类时，我们会基于训练集去计算各分类标签的后验概率，然后进行相互地比较，取其中值最大的标签就是对应的类别。

当然，在使该式能够解决分类问题前，还需要假设：相对于各类别标签，特征之间是相互独立的，所以有下式成立：

$$P(x_1, x_2, \dots, x_d | c) = \prod_{i=1}^d P(x_i | c) \quad (3.2)$$

针对“离散数据”的朴素贝叶斯，那么就是正常的用频率代替概率的计算，比较简单则不再赘述。针对“连续数据”的朴素贝叶斯则通常采用高斯分布假设：

$$P(x_i | c) = \frac{1}{\sqrt{2\pi\sigma_{ic}^2}} \exp\left(-\frac{(x_i - \mu_{ic})^2}{2\sigma_{ic}^2}\right) \quad (3.3)$$

对于每个类分别估计均值和方差，预测时仍是：

$$P(c) \prod_i P(x_i | c) \quad (3.4)$$

优点：

- A. **计算效率极高**：训练为线性时间，适合大规模数据。
- B. **稳定性好**：高维稀疏数据（文本）效果非常强。
- C. **可解释性强**：每个特征对分类结果贡献可直接查看。
- D. **对噪声不敏感**：概率模型本身具备一定鲁棒性。

缺点：

- A. 独立性假设过强：现实中特征往往相关，导致模型不是最优。
- B. 对数值关系的表达能力弱（除非采用复杂分布假设）。
- C. 对连续特征分布假设较强（Gaussian NB 未必贴合实际分布）。
- D. 对出现概率为 0 的项需平滑，否则会导致预测概率为 0。

3.2 决策树

决策树的核心原理简单来说就是：不断选择“最能区分数据的特征”把数据一层层分开，直到每个节点的样本几乎属于同一类。本质上来说就是找一个特征把数据分得最干净，再对每一部分重复这个分法，最终形成一棵“如果…那么…”的树。所以这棵树具体怎么构建呢？

- A. 从所有特征中选择一个最优特征进行划分，使得划分后子集的“纯度提升最大”
- B. 对每个子节点重复步骤 1（递归）
- C. 满足停止条件后生成叶节点

那么如何量化这个纯度呢？不同的决策树构建算法对纯度的度量不同，它们主要分为以下几种：

A. 信息熵 (Entropy)

作用：衡量数据混乱程度。越混乱熵越高。越纯净熵越低。

计算公式如下：

$$Entropy(D) = -\sum p_k \log p_k \quad (3.5)$$

如果节点中类别完全一致 $\rightarrow$ 熵 = 0；如果类别一半一半 $\rightarrow$ 熵较高；类别越均匀越混乱，熵越大。

B. 信息增益 (Information Gain) ——ID3 使用

作用：衡量使用某个特征划分后，数据变“更干净”了多少。

计算公式如下：

$$Gain(D, A) = Entropy(D) - \sum_v \frac{|D_v|}{|D|} Entropy(D_v) \quad (3.6)$$

差值越大，说明该特征让数据变更纯 $\rightarrow$ 特征更好。

C. 信息增益率 (Gain Ratio) ——C4.5 使用

作用：由于信息增益会偏向取值个数多的特征，因此 C4.5 用增益率来调整偏差。

计算公式如下：

$$Gainratio = \frac{Gain(D,A)}{IV(A)} \quad (3.7)$$

其中 $IV(A)$ 是“特征本身的取值散布程度”（值越多， IV 越大）→ 防止倾向大量类别的特征。

D. 基尼指数 (Gini) ——CART 使用

计算公式如下：

$$Gini(D) = 1 - \sum p_k^2 \quad (3.8)$$

数值越小表示节点越纯，CART 选择能让 Gini 下降最多的划分。

对于特征是连续的，比如：年龄、收入、温度，决策树则会将每一个可能的“分割点”作为候选，而后把样本分成两部分：大于阈值与小于阈值两部分，而后根据该划分计算信息增益或基尼指数，而后选最优阈值。最终获得最优特征和最优切分点（阈值）。

优点：

- A. 可解释性极强：可直接可视化树结构。
- B. 能处理离散和连续数据。
- C. 对数据形态无强假设（非线性关系可捕捉）。
- D. 训练和预测速度快。

缺点：

- A. 容易过拟合（尤其未剪枝时）。
- B. 决策边界是轴对齐的矩形，不够光滑。
- C. 数据的小幅波动可能引起树结构巨大变动（不稳定）。
- D. 不擅长高维稀疏空间（文本）。

3.3 K 最近邻算法

K 最近邻算法的核心思想就是：一个样本属于哪一类，就看离它最近的 K 个已知样本里多数是什么类。它不显式训练模型，而是在预测时基于距离度量做“多数表决”，因此属于典型的 lazy learning。

假设 k 是最近邻数目， D 是训练样例的集合，给定输入样本 x ，计算 x 和每个训练样例之间的距离 d ，选择离 x 最近的 k 个训练样例的集合 $D_z \subseteq D$ ，按下式找出 x 周围 k 个样例中的多数类别，将该类别赋给 x ：

$$y' = \operatorname{argmax} \sum_{(x_i, y_i) \in D_z} I(y = y_i) \quad (3.9)$$

距离度量可以选择欧式距离、曼哈顿距离等等常用的距离度量公式。

k 值也会相应影响分类的结果， k 如果过小，决策边界复杂，易过拟合； k 如果过大，则容易被噪声抑制，但边界变平滑。

优点：

- A. 模型形式非常简单，易理解。
- B. 对数据分布无需假设，适合多模态分布。
- C. 可用于分类、回归、多标签任务。

缺点：

- A. 预测阶段计算成本高（必须计算与所有训练样本距离）。
- B. 对高维空间非常不友好（维度灾难）。
- C. 对特征尺度敏感，需要标准化/归一化。
- D. 没有显式模型，无法解释特征重要性。

4. 源代码及实现结果

4.1 源代码分析

(1) 数据加载与预处理

该函数是整个实验的数据入口。它首先使用 `pandas` 读取指定路径下的 `SMSSpamCollection` 数据集，由于原始数据没有表头且以制表符分隔，函数进行了相应的格式处理。接着，使用 `LabelEncoder` 将文本形式的标签（"ham"和"spam"）转换为计算机可计算的数值标签（0 和 1）。核心步骤是利用 `TfidfVectorizer` 将非结构化的短信文本转化为高维的 TF-IDF 向量矩阵，并特意限制了 `max_features=3000` 以在保留主要语义的同时降低计算维度，从而优化 KNN 算法的运行速度。最后，函数将处理好的数据按照 7:3 的比例划分为训练集和测试集，为后续模型训练做准备。


```

01 # =====
02 # 1. 数据加载与预处理
03 # =====
04 def load_and_preprocess_data(path):
05     print(f"正在读取数据集: {path} ...")
06     try:
07         # SMSSpamCollection 通常没有表头，第一列是标签，第二列是短信内容，用制表
08         符分隔
09         df = pd.read_csv(path, sep='\t', header=None, names=['label', 'message'])
10     except FileNotFoundError:
11         print(f"错误: 找不到文件 {path}。请确保文件路径正确。")
12         exit()
13
14     # 标签编码: ham -> 0, spam -> 1
15     le = LabelEncoder()
16     y = le.fit_transform(df['label'])
17
18     # 文本特征提取 (TF-IDF)
19     print("正在进行 TF-IDF 向量化 ...")
20     tfidf = TfidfVectorizer(stop_words='english', max_features=3000) # 限制特征数以加快
21     KNN 速度
22     X = tfidf.fit_transform(df['message']).toarray() # KNN 通常需要稠密矩阵或特定稀疏处
23     理，这里转 array 方便处理
24
25     # 划分训练集和测试集
26     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
27
28     return X_train, X_test, y_train, y_test

```

(2) 模型定义与评测流程

这是实验的核心逻辑部分。函数首先定义了一个包含三个待测模型（多项式朴素贝叶斯、决策树、K 最近邻）的字典。随后，它通过循环依次对每个模型执行完整的“训练-预测-评估”流程：

1. 训练与计时：调用 `fit` 方法训练模型，并记录训练耗时。
2. 预测与计时：调用 `predict` 方法对测试集进行分类，并记录预测耗时；同时获取预测概率（`predict_proba`），用于后续绘制 ROC 和 PR 曲线。
3. 指标计算：分别计算准确率（Accuracy）、精确率（Precision）、召回率（Recall）和 F1 分数。

最终，函数将所有模型的评估指标、预测结果和时间开销封装在一个字典中返回，

供可视化模块使用。

```
01 # =====
02 # 2. 模型定义与评测流程
03 # =====
04 def evaluate_models(X_train, X_test, y_train, y_test):
05     # 定义要评测的模型
06     models = {
07         'Naive Bayes': MultinomialNB(),
08         'Decision Tree': DecisionTreeClassifier(random_state=42),
09         'KNN': KNeighborsClassifier(n_neighbors=5)
10     }
11
12     results = {}
13
14     print("\n 开始训练与评测模型...")
15     print("-" * 80)
16     print(f'{'Model':<15} {'Acc':<10} {'Prec':<10} {'Recall':<10} {'F1':<10} {'Train Time(s)':<15} {'Pred'
17 Time(s)':<15}}')
18     print("-" * 80)
19
20     for name, model in models.items():
21         # 1. 训练并计时
22         start_train = time.time()
23         model.fit(X_train, y_train)
24         end_train = time.time()
25         train_time = end_train - start_train
26
27         # 2. 预测并计时
28         start_pred = time.time()
29         y_pred = model.predict(X_test)
30         end_pred = time.time()
31         pred_time = end_pred - start_pred
32
33         # 获取概率用于绘制 ROC 和 PR 曲线
34         if hasattr(model, "predict_proba"):
35             y_prob = model.predict_proba(X_test)[: , 1]
36         else:
37             # 对于没有 predict_proba 的模型（很少见），使用 decision_function 或 0/1
38             y_prob = y_pred
39
40         # 3. 计算指标
41         acc = accuracy_score(y_test, y_pred)
42         prec = precision_score(y_test, y_pred)
43         rec = recall_score(y_test, y_pred)
```

```

44         f1 = f1_score(y_test, y_pred)
45
46         # 打印结果
47         print(f'{name:<15}
48         {acc:.4f}      {prec:.4f}      {rec:.4f}      {f1:.4f}      {train_time:.4f}      {pred_time:.4f}')
49
50         # 存储结果用于可视化
51         results[name] = {
52             'metrics': [acc, prec, rec, f1], # 用于雷达图
53             'y_prob': y_prob,                # 用于 ROC/PR
54             'y_test': y_test,
55             'train_time': train_time,
56             'pred_time': pred_time
57         }
58
59     return results

```

(3) 可视化函数

这一部分包含三个独立的绘图函数，旨在从不同维度展示模型性能：

- **plot_radar_chart (雷达图)**：将 Accuracy、Precision、Recall、F1-Score 这四个指标映射到极坐标系上，直观地对比不同模型在各项能力上的均衡程度和综合表现。
- **plot_roc_auc (ROC 曲线)**：绘制真阳性率 (TPR) 与假阳性率 (FPR) 的关系曲线，并计算 AUC 值，用于评估模型在不同阈值下的分类能力，特别是模型区分正负样本的总体可靠性。
- **plot_pr_auc (PR 曲线)**：绘制精确率与召回率的关系曲线。针对垃圾短信这种类别不平衡（垃圾短信通常较少）的数据集，PR 曲线比 ROC 曲线更能真实地反映模型对少数类（垃圾短信）的识别效果。

```

01 # =====
02 # 3. 可视化函数
03 # =====
04
05 def plot_radar_chart(results):
06     """绘制雷达图：多指标综合比较"""
07     categories = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
08     N = len(categories)
09
10     # 计算角度
11     angles = [n / float(N) * 2 * pi for n in range(N)]
12     angles += angles[:1] # 闭合回路
13
14     plt.figure(figsize=(8, 8))
15     ax = plt.subplot(111, polar=True)
16
17     # 设置刻度
18     plt.xticks(angles[:-1], categories, color='grey', size=12)
19     ax.set_rlabel_position(0)
20     plt.yticks([0.2, 0.4, 0.6, 0.8, 1.0], ["0.2", "0.4", "0.6", "0.8", "1.0"], color="grey", size=7)
21     plt.ylim(0, 1)
22
23     colors = ['b', 'r', 'g']
24
25     for i, (name, data) in enumerate(results.items()):
26         values = data['metrics']
27         values += values[:1] # 闭合
28
29         ax.plot(angles, values, linewidth=1, linestyle='solid', label=name, color=colors[i])
30         ax.fill(angles, values, color=colors[i], alpha=0.1)
31
32     plt.title('各模型性能指标雷达图', size=16, y=1.1)
33     plt.legend(loc='upper right', bbox_to_anchor=(0.1, 0.1))
34
35 def plot_roc_auc(results):
36     """绘制多模型 ROC-AUC 曲线"""
37     plt.figure(figsize=(8, 6))
38
39     for name, data in results.items():
40         y_test = data['y_test']
41         y_prob = data['y_prob']
42
43         fpr, tpr, _ = roc_curve(y_test, y_prob)
44         roc_auc = auc(fpr, tpr)

```

```

45
46     plt.plot(fpr, tpr, lw=2, label=f'{name} (AUC = {roc_auc:.3f})')
47
48     plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
49     plt.xlim([0.0, 1.0])
50     plt.ylim([0.0, 1.05])
51     plt.xlabel('False Positive Rate (FPR)')
52     plt.ylabel('True Positive Rate (TPR)')
53     plt.title('多模型 ROC 曲线对比')
54     plt.legend(loc="lower right")
55     plt.grid(alpha=0.3)
56
57 def plot_pr_auc(results):
58     """绘制多模型 PR-AUC 曲线 (Precision-Recall)"""
59     plt.figure(figsize=(8, 6))
60
61     for name, data in results.items():
62         y_test = data['y_test']
63         y_prob = data['y_prob']
64
65         precision, recall, _ = precision_recall_curve(y_test, y_prob)
66         avg_prec = average_precision_score(y_test, y_prob)
67
68         plt.plot(recall, precision, lw=2, label=f'{name} (AP = {avg_prec:.3f})')
69
70     plt.xlabel('Recall')
71     plt.ylabel('Precision')
72     plt.title('多模型 Precision-Recall 曲线对比')
73     plt.legend(loc="lower left")
74     plt.grid(alpha=0.3)

```

(4) 主程序

这是脚本的执行控制流。它将上述所有模块串联起来：首先定义数据集路径，调用数据预处理函数获取训练测试数据；然后将数据传入评估函数，获得所有模型的评测结果；最后依次调用三个可视化函数生成图表，并额外绘制了一个柱状图来对比不同模型的训练与预测时间开销。程序运行结束时会弹出包含所有分析图表的窗口，完成整个实验的汇报展示。

```

01 # =====
02 # 主程序
03 # =====
04 if __name__ == "__main__":
05     # 数据集路径
06     DATA_PATH = "./dataset/SMSSpamCollection"
07
08     # 1. 准备数据
09     X_train, X_test, y_train, y_test = load_and_preprocess_data(DATA_PATH)
10
11     # 2. 评测模型
12     results = evaluate_models(X_train, X_test, y_train, y_test)
13
14     # 3. 可视化分析
15     print("\n 正在生成可视化图表...")
16
17     plot_radar_chart(results)
18     plot_roc_auc(results)
19     plot_pr_auc(results)
20
21     # 也可以绘制时间对比图（额外赠送）
22     plt.figure(figsize=(8, 5))
23     times_train = [results[m]['train_time'] for m in results]
24     times_pred = [results[m]['pred_time'] for m in results]
25     models_names = list(results.keys())
26
27     x = np.arange(len(models_names))
28     width = 0.35
29
30     plt.bar(x - width/2, times_train, width, label='Training Time')
31     plt.bar(x + width/2, times_pred, width, label='Prediction Time')
32     plt.xticks(x, models_names)
33     plt.ylabel('Time (seconds)')
34     plt.title('模型时间开销对比')
35     plt.legend()
36
37     print("可视化完成，请查看弹出的窗口。")
38     plt.show()

```

4.2 结果与分析

Model	Acc	Prec	Recall	F1	Train Time(s)	Pred Time(s)
Naive Bayes	0.9844	0.9950	0.8884	0.9387	0.0320	0.0100
Decision Tree	0.9677	0.9126	0.8393	0.8744	7.7485	0.0128
KNN	0.9097	1.0000	0.3259	0.4916	0.0066	1.1045

图 1 评测结果表
各模型性能指标雷达图

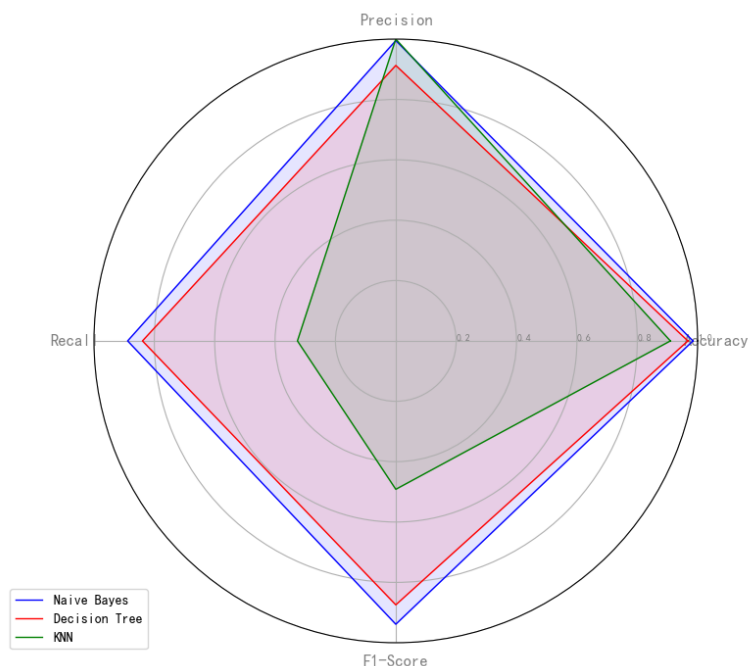


图 2 结果雷达图

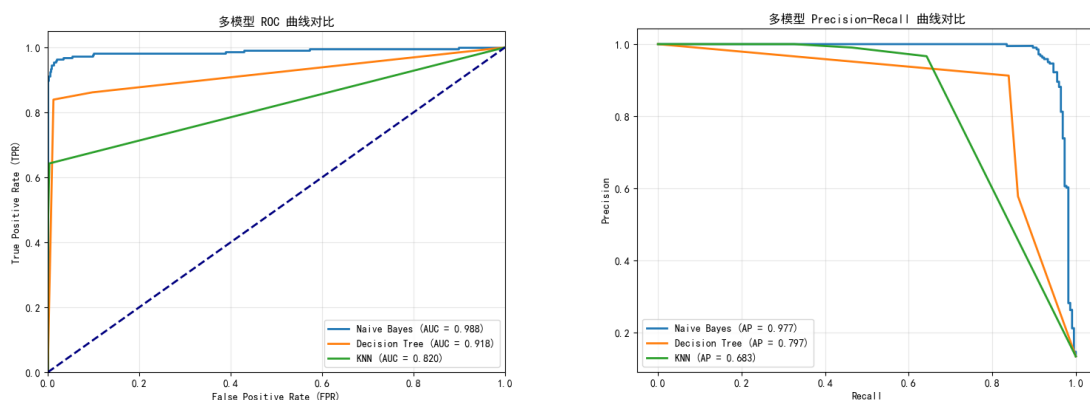


图 3 评测结果 ROC 与 PR 图

可以从最终评测表和雷达图中，我们可以看出朴素贝叶斯 (Naive Bayes) 模型表现出压倒性的优势，无论是在分类精度、综合评价指标 (F1-Score、AUC)，还是在计算效率 (时间开销) 上，都优于决策树和 K 最近邻 (KNN)。

朴素贝叶斯的 Accuracy (0.9844) 和 F1-Score (0.9387) 均为最高；Precision

(0.9950) 极高，意味着几乎没有“误杀”（将正常短信误判为垃圾短信）；Recall (0.8884) 表现良好，能拦截近 90% 的垃圾短信；ROC 曲线的 $AUC = 0.988$ ，曲线最贴近左上角，说明模型对正负样本的区分能力极强；PR 曲线的 $AP = 0.977$ ，曲线下面积最大，说明在类别不平衡的情况下（垃圾短信较少），该模型依然能保持高查准率和高查全率。

决策树表现得中规中矩，各项指标均低于朴素贝叶斯。Accuracy (0.9677) 尚可，但 F1 (0.8744) 下降较明显，相比朴素贝叶斯，决策树的误判率略高，且漏判率也更高；另外，决策树的训练时间是朴素贝叶斯 (0.032s) 的 240 倍以上，分析原因可能是 TF-IDF 生成了 3000 个特征，决策树在构建过程中，每个节点分裂都需要遍历所有特征以计算信息增益，计算量巨大；ROC 和 PR 曲线呈现出明显的“折线”形状，且覆盖面积小于朴素贝叶斯。这反映了决策树作为判别模型，在未剪枝或深度受限时，输出的概率不够平滑。

K 最近邻算法则严重偏斜，综合效果最差。它的评测结果很极端，Precision 是 1.0，这看起来很完美，意味着凡是它认为是垃圾短信的，全部真的是垃圾短信，完全没有误杀；但是 recall 极其低，表示它只能识别出 32% 的垃圾短信，漏判了近 70%。导致最终 F1-Score (0.4916) 极低；并且它的预测时间最慢，是朴素贝叶斯的 100 倍，分析原因可能在于 KNN 是惰性学习，预测时需要计算新样本与训练集中数千个样本的距离，计算量随数据量线性增长，这也与前文分析的原理吻合。

5. 个人体会

通过本次关于朴素贝叶斯、决策树与 KNN 三类文本分类算法的系统性实验，我对监督学习模型在高维稀疏数据场景下的表现差异有了更深刻的认识。首先，本次实验让我意识到算法的理论假设与数据特征之间的匹配度对于最终效果影响极大。朴素贝叶斯尽管在理论上依赖强独立性假设，但在文本分类这种特征高度稀疏且维度巨大的任务中反而能够展现出非常稳定而高效的性能，这一点在实验结果中得到了明确验证。相比之下，决策树虽然具有可解释性强的优势，但面对成千上万维度的 TF-IDF 特征，其计算复杂度迅速攀升，模型结构也容易受到噪声影响。KNN 的问题则更加典型——高维度导致距离度量失真，使其在文本任务中几乎不具备实用价值。本次实验让我对分类问题中的经典算法有了深刻的理解，这对我今后的研究和学习都有着全面的帮助。